

Padrão Observer

O padrão observer serve para criar sistemas orientados a eventos, onde as ações de um objeto disparam reações automáticas em outros.

Problema:

Imagine que você está desenvolvendo uma plataforma de streaming. Você tem a classe Canal e a classe Usuario. Os usuários estão muito interessados nos vídeos de um canal específico.

Se você tentar resolver isso sem um padrão, restam duas alternativas ruins:

1. Consulta Constante (Polling): O Usuario fica checando o Canal a cada 5 minutos para ver se há vídeos novos. Isso é um desperdício massivo de processamento, memória e rede (o drone/servidor ficaria sobrecarregado com requisições inúteis).
2. Acoplamento Rígido: O Canal guarda uma lista direta de objetos Usuario do tipo exato da classe. Toda vez que um vídeo sai, o canal varre a lista chamando cada usuário. Se amanhã você quiser notificar uma SmartTV ou uma EmpresaDeMarketing em vez de um usuário comum, terá que modificar o código interno do Canal, quebrando o princípio de responsabilidade única.

Solução:

O padrão Observer sugere que você crie um mecanismo de assinatura (Publisher/Subscriber).

O objeto que possui o estado interessante (neste caso, o Canal) é chamado de Sujeito (Subject). Os objetos que querem saber quando o estado muda são chamados de Observadores (Observers).

O Sujeito mantém uma lista de ponteiros para uma interface genérica de observador. Qualquer classe que implemente essa interface pode se "inscrever" ou "cancelar a inscrição" a qualquer momento. Quando o evento principal acontece, o Sujeito simplesmente percorre a lista notificando todo mundo através de um método padrão.

Código Observer

O código implementa o padrão de projeto Observer em C++. Neste caso, o observer notifica um usuário do youtube.

Ele está dividido em três arquivos: StreamingEventos.h, StreamingEventos.cpp e main.cpp.

Para executar os arquivos existe um Makefile com os comandos make (para compilar e rodar o código) e make clean para deletar o executável após compilação.

1. Contrato dos Observadores (StreamingEventos.h)

Para que o canal consiga notificar as pessoas, ele precisa de uma forma genérica de conversar com elas. É aí que entra a Interface:

- InterfaceInscrito (O contrato): Possui o método virtual puro atualizar(). Qualquer classe que queira receber notificações do canal precisa herdar dessa interface e dizer o que fará quando for atualizada.
- UsuarioYoutube (O observador concreto): É o usuário real. Ele implementa o método atualizar() definindo que, ao receber o alerta, vai printar uma notificação simulando a tela do celular.

2. Sujeito Monitorado (StreamingEventos.cpp)

A classe CanalYoutube é o nosso Subject (ou Publisher). Ela guarda dentro de um vetor (std::vector<InterfaceInscrito*>) a lista de todo mundo que clicou no "sininho".

Ela gerencia essa lista com três métodos fundamentais:

1. inscrever(): Adiciona um ponteiro de usuário ao vetor.
2. desinscrever(): Procura o usuário na lista e o remove usando o algoritmo std::remove.
3. notificarInscritos(): O coração do padrão. Um loop for simples que passa de inscrito em inscrito disparando o método atualizar().

Quando a lógica de negócio acontece em publicarVideo(), o canal altera seu estado interno e imediatamente dispara o notificarInscritos().

3. Dinâmica em Tempo de Execução (main.cpp)

No arquivo main.cpp, podemos ver o quão flexível e dinâmico esse padrão é:

1. O canal "Mundo Tech" é criado.
2. Alice, Bob e Carlos se inscrevem.
3. O canal posta um vídeo sobre C++. Internamente, o código roda o loop de notificação e os três recebem o alerta no celular.

4. Carlos decide se desinscrever (`canalTech->desinscrever(carlos);`). Ele é removido do vetor do canal em tempo de execução.
5. O canal posta um segundo vídeo. Dessa vez, o loop só encontra a Alice e o Bob na lista. Carlos não é incomodado.

Link para o código do site refactoring.guru utilizado como exemplo:

<https://refactoring.guru/pt-br/design-patterns/observer/cpp/example>